Initiation au Développement : Variables, Mémoire RAM, Structures Conditionnelles et Répétitives

Thème principal : Algorithmique et Programmation Impérative

Synthèse exécutive : Ce cours pose les fondations absolues de la programmation informatique pour les étudiants en BUT Informatique (IUT de Bayonne et du Pays Basque, Campus d'Anglet). L'objectif est d'apprendre à formaliser la résolution d'un problème métier sous forme d'instructions rigoureuses et exécutables par une machine. La séance couvre l'allocation et le typage de la mémoire vive (RAM) à travers les variables et l'affectation destructive, l'aiguillage du flux d'exécution par les structures conditionnelles (SI / ALORS / SINON) basées sur la logique booléenne, ainsi que l'automatisation des répétitions via les structures itératives (boucles POUR et TANT QUE), tout en prévenant les erreurs critiques de conception comme les boucles infinies.

1. Les Variables, le Typage et la Représentation en Mémoire RAM

En programmation impérative, le processeur exécute des instructions en manipulant des données conservées temporairement dans la mémoire vive (RAM). Pour éviter de devoir gérer manuellement des adresses hexadécimales complexes (par exemple 0x7FFF5FBFF058), l'algorithmique utilise la notion d'abstraction de la variable. Déclarer une variable consiste à réserver une plage d'octets précise en mémoire. Le type indique au compilateur ou à l'interprète le format du codage binaire : par exemple, un Entier standard utilise généralement 4 octets (32 bits), tandis qu'un Réel double précision requiert 8 octets. L'opération d'affectation respecte un ordre séquentiel strict : le membre de droite est intégralement évalué, puis le résultat obtenu est écrit dans la case mémoire associée à la variable de gauche. Cette écriture est destructive : l'ancienne valeur contenue dans la mémoire est définitivement perdue.

- Variable : Emplacement mémoire nommé (adresse abstraite dans la RAM) permettant d'emmagasiner et de modifier une donnée au cours de l'exécution.
- Typage : Spécification explicite du format de la donnée (Entier, Réel, Chaîne, Booléen) fixant sa taille d'allocation en octets et la plage d'opérations valides.
- Affectation (<- ou =) : Opération destructive consistant à évaluer l'expression située à droite puis à écraser la valeur courante de la variable située à gauche.

Cas concret enseignant : *Le professeur compare la mémoire RAM à un grand meuble à tiroirs : déclarer la variable 'age : Entier', c'est coller une étiquette 'age' sur un tiroir spécifique et fabriquer ce tiroir à la dimension exacte pour recevoir uniquement un nombre entier. Y placer une nouvelle valeur chasse et détruit instantanément la valeur précédente.*

2. Les Structures Conditionnelles et la Logique Booléenne

Par défaut, un programme informatique s'exécute de manière séquentielle, ligne par ligne. Les structures conditionnelles rompent cette séquentualité en offrant des branchements conditionnés par l'état des variables ou les saisies des utilisateurs. Le processeur évalue d'abord la condition booléenne : si celle-ci renvoie VRAI, le bloc 'ALORS' est exécuté et le bloc 'SINON' est ignoré. Si elle renvoie FAUX, le processeur saute directement au bloc 'SINON' (s'il existe) ou poursuit la suite du programme. Lors de la conception algorithmique, il est indispensable de vérifier l'exhaustivité des cas pour s'assurer qu'aucune situation limite ne reste non traitée par le système.

- Condition Booléenne : Expression logique dont le résultat est exclusivement VRAI (True) ou FAUX (False).
- Bloc SI / ALORS / SINON : Aiguillage du flux d'instructions dérivant l'exécution selon le résultat du test booléen.
- Opérateurs Logiques (ET, OU, NON) : Connecteurs permettant d'associer plusieurs conditions simples pour formaliser des règles métiers complexes.

Cas concret enseignant : *Exemple du système de contrôle de la borne d'accès du parking du campus de Bayonne/Anglet : Si (carte_etudiant_valide == VRAI ET parking_complet == FAUX) ALORS lever la barrière ; SINON afficher le message d'erreur 'Accès Refusé' sur l'écran.*

3. Les Structures Répétitives (Boucles POUR et TANT QUE)

Les boucles évitent la duplication de code en permettant de répéter un même bloc d'instructions. Le choix de la structure dépend directement du problème : si le nombre d'itérations est fixe et connu au départ (ex: parcourir une liste de 500 étudiants), la boucle POUR est obligatoire car elle gère l'incrémentation automatique du compteur. En revanche, si la répétition dépend d'un événement dynamique (ex: attendre la saisie d'un mot de passe valide), la boucle TANT QUE doit être privilégiée. Dans une boucle TANT QUE, au moins une variable engagée dans la condition de test doit obligatoirement être modifiée à l'intérieur du corps de la boucle, sous peine de bloquer indéfiniment l'application.

- Boucle POUR (FOR) : Structure itérative déterministe avec compteur automatique, réservée aux cas où le nombre d'itérations est fixe et connu avant le début de la boucle.
- Boucle TANT QUE (WHILE) : Structure itérative conditionnelle, exécutée tant qu'une condition logique reste vraie, adaptée aux répétitions indéterminées.
- Boucle Infinie : Dysfonctionnement critique se produisant lorsque la condition de sortie d'une boucle TANT QUE ne devient jamais fausse, bloquant indéfiniment le programme.

Cas concret enseignant : *Le bug du distributeur automatique de billets (DAB) : si l'algorithme oublie de remettre la variable 'carte_insérée' à FAUX après l'éjection physique de la carte, la boucle TANT QUE demandant la saisie du code PIN tourne indéfiniment et bloque le distributeur.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi doit-on obligatoirement spécifier le type d'une variable dans des langages comme C ou C++ alors que ce n'est pas nécessaire en Python ?
Réponse du professeur : En C ou C++, le typage statique permet au compilateur de réserver exactement la bonne quantité d'octets en mémoire et de générer du code machine optimisé. En Python, le typage est dynamique : l'interpréteur gère les types à la volée au détriment des performances d'exécution.

Question étudiant : Quelle est la différence exacte entre l'opérateur '=' et l'opérateur '==' ?
Réponse du professeur : L'opérateur '=' est l'affectation (il évalue la droite et stocke le résultat à gauche). L'opérateur '==' est un test d'égalité logique qui renvoie un booléen (VRAI ou FAUX) sans modifier les variables comparées.

■ Exercice & Démonstration d'Application

Énoncé : Écrire un algorithme complet en pseudo-code qui demande à l'utilisateur de saisir 5 notes (sur 20), calcule la moyenne arithmétique de ces notes, puis affiche 'Admis' si la moyenne est supérieure ou égale à 10.0, ou 'Ajourné' dans le cas contraire.

Résolution pas à pas :
1. Déclaration et initialisation des variables : - somme : Réel <- 0.0 (accumulateur initialisé à 0) - note : Réel (stocke la saisie courante) - moyenne : Réel (stocke le résultat final) - i : Entier (compteur de la boucle) 2. Saisie des notes et accumulation (Le nombre de notes est connu = 5 -> Boucle POUR) : POUR i ALLANT DE 1 À 5 FAIRE Afficher "Entrez la note numéro ", i, " : " Saisir note somme <- somme + note FIN POUR 3. Calcul de la moyenne arithmétique : moyenne <- somme / 5.0 4. Test conditionnel et affichage du statut : SI moyenne >= 10.0 ALORS Afficher "Résultat : Admis (Moyenne = ", moyenne, ")" SINON Afficher "Résultat : Ajourné (Moyenne = ", moyenne, ")" FIN SI

Résultat : L'algorithme s'exécute de manière déterministe en exactement 5 itérations. La variable 'somme' cumule chaque note saisie sans réinitialisation intempestive dans la boucle. La moyenne réelle est calculée puis testée rigoureusement par la structure conditionnelle pour afficher la décision finale.

■ Auto-Évaluation Flash d'Examen

Q : Que se passe-t-il si la condition d'une boucle TANT QUE est évaluée à FAUX dès le premier contrôle ?

R : Le corps de la boucle n'est jamais exécuté (0 itération).

Q : Quelle est la règle stricte pour choisir entre une boucle POUR et une boucle TANT QUE ?

R : La boucle POUR s'utilise lorsque le nombre d'itérations est connu à l'avance (déterministe), tandis que le TANT QUE s'utilise pour des conditions dynamiques ou indéterminées.

Q : Pourquoi est-il dangereux de comparer deux nombres réels (flottants) directement avec l'opérateur d'égalité '==' ?

R : En raison des imprécisions de représentation des nombres à virgule flottante dans la norme IEEE 754, deux calculs théoriquement égaux peuvent différer d'une infime fraction.

Fiche académique générée et vérifiée par Kai pour Matéi • Hébergée sur http://88.96.47.180:8765